THE ICT UNIVERSITY

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGIES

SPRING 2026
SEN3244 – SOFTWARE ARCHITECTURE
FINAL EXAMINATION PROJECT

PROJECT REPORT

TasteCam-Heritage

A Platform for Preserving and Discovering Culinary Heritage

Course Code	SEN3244 – Software Architecture
Project Topic	TasteCam-Heritage
Student Name	Abomo Mvogo Therese Damaris
Registration Number	ICTU20241280
GitHub Repository	https://github.com/abomomvogoth/TasteCam-Heritage

Instructor: Engr. Tekoh Palma Date: June 2026

CHAPTER ONE: INTRODUCTION

1.1 General Introduction

TasteCam-Heritage is a web-based application designed to digitize, preserve, and promote culinary heritage. Food is one of the most powerful expressions of culture and identity. However, traditional recipes, cooking techniques, and culinary knowledge risk being lost as communities modernize. TasteCam-Heritage addresses this challenge by providing a platform where users can discover, share, and archive traditional recipes from diverse cultural backgrounds, with a particular focus on African and Cameroonian culinary traditions.

The application follows a modern software architecture combining a RESTful Node.js/Express backend, a PostgreSQL database, and a dynamic frontend. The system is fully containerized using Docker and orchestrated with Kubernetes, deployed through an automated CI/CD pipeline powered by Jenkins, and monitored using Prometheus and Grafana.

1.2 Aim and Objectives

The main aim of this project is to design, build, and deploy a full-stack web application that showcases best practices in modern software architecture.

The specific objectives are:

- To implement a layered RESTful API architecture using Node.js and Express.js
- To containerize the application using Docker and orchestrate it with Kubernetes
- To automate the build, test, and deployment cycle using a Jenkins CI/CD pipeline
- To ensure code quality through automated testing with Jest, achieving at least 80% code coverage
- To monitor system performance using Prometheus and Grafana dashboards
- To manage infrastructure configuration using Ansible playbooks
- To apply Scrum methodology throughout the development process

1.3 Problem Statement

Culinary heritage represents centuries of cultural knowledge, yet no centralized digital platform exists in Cameroon or Central Africa to archive and share traditional recipes. Existing global platforms (such as YouTube or food blogs) lack structured metadata, cultural context, and offline-friendly access. TasteCam-Heritage fills this gap by providing a structured, searchable, and community-driven platform for culinary heritage preservation, built on robust and scalable software architecture principles.

CHAPTER TWO: LITERATURE REVIEW

2.1 Software Development Methodologies

Software development methodologies are structured approaches to planning, building, and managing software projects. The most widely adopted methodologies include the Waterfall model, Agile, Scrum, Kanban, and DevOps practices. Each methodology offers trade-offs between flexibility, speed, documentation, and predictability.

2.2 Comparison between Software Development Methodologies

Methodology	Strengths	Weaknesses
Waterfall	Clear phases, good documentation	Inflexible, late testing
Agile	Flexible, iterative, customer-focused	Requires active client involvement
Scrum	Sprint-based, defined roles, fast delivery	Requires experienced Scrum Master
Kanban	Visual workflow, continuous delivery	No defined iterations

2.3 Reason for Choosing Scrum Methodology

Scrum was chosen for TasteCam-Heritage because of its iterative nature and suitability for projects where requirements may evolve. As a solo developer managing multiple technical domains (backend, frontend, DevOps), Scrum provided a structured yet flexible framework. Sprint planning allowed prioritization of features, while retrospectives helped identify and resolve bottlenecks efficiently.

2.4 Review of Related Concepts

Microservices architecture decomposes applications into small, independently deployable services. However, for a project of this scope, a Layered (N-Tier) Architecture was more appropriate, offering simplicity, clear separation of concerns, and easier maintenance.

Containerization with Docker provides environment consistency across development, testing, and production. Kubernetes extends this by managing container lifecycles, scaling, and self-healing. Together, they form the backbone of modern cloud-native application deployment.

2.5 Review of Related Literature

Several platforms inspired the design of TasteCam-Heritage. TasteAtlas (2018) maps traditional food globally but lacks user contribution features. Open Food Facts provides structured food data but focuses on nutrition rather than cultural context. TasteCam-Heritage distinguishes itself by combining a recipe archive with cultural storytelling and a RESTful API for future mobile integration.

CHAPTER THREE: METHODOLOGY AND MATERIALS

3.1 Research Methodology

This project adopted an experimental and applied research methodology. The development process was guided by agile principles (Scrum), ensuring iterative delivery of functional software. Each sprint produced working software increments, with continuous integration validating code quality at every stage.

3.2 System Requirements

Functional Requirements:

- Users can browse, search, and filter traditional recipes by region or ingredient
- Users can submit new recipes with images and cultural descriptions
- The system exposes a RESTful API for all CRUD operations on recipes
- API documentation is available through Swagger UI
- The system supports user authentication (future sprint)

Non-Functional Requirements:

- Performance: API response time under 200ms for standard queries
- Scalability: Kubernetes horizontal pod autoscaling for traffic spikes
- Availability: 99.9% uptime target through rolling updates
- Security: Environment variables for secrets management via .env and dotenv
- Maintainability: 80%+ code coverage enforced by Jest in CI/CD pipeline

3.3 System Design

3.3.1 Architecture of the System (HLD)

TasteCam-Heritage follows a Layered Architecture (also known as N-Tier Architecture) with three main layers: the Presentation Layer (frontend), the Business Logic Layer (Express.js API), and the Data Layer (PostgreSQL database). This architecture promotes separation of concerns and facilitates independent development and testing of each layer.

Key architectural components:

- Frontend: Serves the user interface on port 4000, communicating with the backend via HTTP REST calls
- Backend API: Node.js + Express.js application on port 3000, handling all business logic and database operations
- Database: PostgreSQL (pg driver), storing recipes, categories, and user data
- Containerization: Docker images for both frontend and backend services
- Orchestration: Kubernetes Deployments with 2 replicas, RollingUpdate strategy, and health probes
- CI/CD: Jenkins pipeline (Checkout → Install Dependencies → Test → Build → Deploy)
- Monitoring: Prometheus metrics collection + Grafana dashboards
- IaC: Ansible playbooks for infrastructure provisioning

3.3.2 UML Diagrams

The system design is represented through the following UML diagrams (see appendix): Use Case Diagram (user interactions with the platform), Class Diagram (Recipe, Category, User entities), Sequence Diagram (API request/response flow), and Deployment Diagram (Docker/Kubernetes infrastructure layout).

3.4 Application of Scrum

3.4.1 Team Organization

Role	Name	Responsibilities
Scrum Master	Abomo Mvogo T. Damaris	Sprint planning, backlog management, obstacle removal
Product Owner	Abomo Mvogo T. Damaris	Defining user stories, acceptance criteria, backlog prioritization
Developer	Abomo Mvogo T. Damaris	Backend, frontend, DevOps, testing, documentation

3.4.2 Workflow Management

The project was divided into two 2-week sprints. Sprint 1 focused on infrastructure setup, backend API development, and database design. Sprint 2 focused on frontend development, CI/CD pipeline configuration, containerization, Kubernetes deployment, monitoring, and documentation.

3.4.3 Scrum Artifacts

Product Backlog (selected items):

ID	User Story	Priority	Status
US-001	As a user, I want to browse recipes by region	High	Done
US-002	As a user, I want to search recipes by ingredient	High	Done
US-003	As a developer, I want automated tests with 80% coverage	High	Done
US-004	As a DevOps, I want CI/CD pipeline with Jenkins	High	Done
US-005	As a DevOps, I want Kubernetes deployment with scaling	High	Done
US-006	As a DevOps, I want Prometheus/Grafana monitoring	Medium	Done
US-007	As a developer, I want Swagger API documentation	Medium	Done
US-008	As a user, I want to submit new recipes	Medium	Sprint 2

3.5 Materials and Technologies

Technology / Tool	Role in the Project
Node.js	JavaScript runtime for backend development
Express.js	Web framework for building the RESTful API
PostgreSQL	Relational database for persistent data storage
pg (node-postgres)	PostgreSQL client driver for Node.js
dotenv	Environment variable management for security
swagger-ui-express	Auto-generated interactive API documentation
Jest	JavaScript testing framework for unit and integration tests
Supertest	HTTP assertion library for API endpoint testing
Docker	Containerization of frontend and backend services
Kubernetes	Container orchestration, scaling, and self-healing
Jenkins	CI/CD pipeline automation (build, test, deploy)
Prometheus	Metrics collection and alerting
Grafana	Dashboard visualization for monitoring metrics
Ansible	Infrastructure as Code for automated provisioning
GitHub	Source code version control and collaboration
Nodemon	Development server with auto-restart on file changes

3.6 CI/CD Pipeline (Jenkins)

The CI/CD pipeline was implemented using Jenkins and is defined in a Jenkinsfile at the root of the repository. The pipeline automates the entire software delivery process from code commit to deployment.

Pipeline stages:

- Stage 1 – Checkout: Pulls the latest code from the GitHub repository (main branch)
- Stage 2 – Install Dependencies: Runs npm install for both backend and frontend directories
- Stage 3 – Backend Tests (Jest + Coverage): Executes Jest test suite with coverage reporting
- Stage 4 – Build: Creates production-ready build artifacts
- Stage 5 – Docker Build: Builds Docker images for frontend and backend services
- Stage 6 – Deploy to Kubernetes: Applies Kubernetes manifests using kubectl

3.7 Infrastructure as Code (Ansible)

Ansible playbooks were written to automate the provisioning and configuration of the VPS infrastructure. At least two playbooks were implemented: one for installing system dependencies (Node.js, Docker, kubectl) and another for starting and configuring services. This ensures reproducible infrastructure setup.

3.8 Containerization and Kubernetes

Both the frontend and backend applications are containerized using Docker. Kubernetes manifests define Deployment resources with 2 replicas for high availability. A RollingUpdate

deployment strategy ensures zero-downtime updates. Readiness and liveness probes monitor container health, and Kubernetes Services expose the applications internally and externally.

3.9 Monitoring (Prometheus & Grafana)

Prometheus is configured to scrape metrics from both the Node.js backend (using prom-client) and the Kubernetes cluster. Grafana dashboards visualize key metrics including API response times, request rates, error rates, CPU and memory usage, and pod availability. Alerting rules are configured to notify on service degradation.

3.10 Test Case Document

ID	Endpoint	Description	Expected Result	Status
TC-001	GET /api/recipes	Returns list of all recipes	200 OK + JSON array	Pass
TC-002	GET /api/recipes/:id	Returns single recipe by ID	200 OK + recipe object	Pass
TC-003	POST /api/recipes	Creates a new recipe	201 Created + new recipe	Pass
TC-004	PUT /api/recipes/:id	Updates an existing recipe	200 OK + updated recipe	Pass
TC-005	DELETE /api/recipes/:id	Deletes a recipe	204 No Content	Pass
TC-006	GET /api/recipes/invalid	Invalid recipe ID	404 Not Found	Pass
TC-007	POST /api/recipes (missing fields)	Validation error	400 Bad Request	Pass

CHAPTER THREE: RESULTS AND DISCUSSIONS

3.1 Application Scenarios

The TasteCam-Heritage platform was successfully deployed and tested. The following scenarios demonstrate the system's functionality:

- Homepage: Displays featured recipes with cultural context and region filters
- Recipe List: Paginated list of all archived recipes, searchable by name or ingredient
- Recipe Detail: Full recipe view with preparation steps, ingredients, and cultural background
- API Explorer: Swagger UI accessible at /api-docs for interactive API testing
- Monitoring Dashboard: Grafana dashboard displaying real-time application and infrastructure metrics

3.2 API Request/Response Examples

GET /api/recipes – Retrieve all recipes:

```
Response: 200 OK
[
  { "id": 1, "name": "Ndole", "region": "Cameroon", "category": "Main Course"
},
  { "id": 2, "name": "Eru", "region": "Cameroon", "category": "Main Course" }
]
```

POST /api/recipes – Create a new recipe:

```
Request Body: { "name": "Koki", "region": "Cameroon", "category": "Side
Dish", "ingredients": [...] }
Response: 201 Created
{ "id": 3, "name": "Koki", "message": "Recipe created successfully" }
```

3.3 Test Coverage Results

The Jest test suite achieved over 80% code coverage across all backend modules. Coverage reports are generated automatically during each Jenkins pipeline run and archived as build artifacts. Supertest was used to perform integration testing directly against the Express.js API routes.

3.4 Kubernetes Deployment Status

The Kubernetes cluster shows the following deployment status:

- tastecam-backend: 2/2 pods running, RollingUpdate strategy active
- tastecam-frontend: 2/2 pods running, readiness and liveness probes passing
- Services: ClusterIP services for internal communication, NodePort/LoadBalancer for external access

CHAPTER FOUR: RECOMMENDATIONS AND CONCLUSION

TasteCam-Heritage successfully demonstrates the application of modern software architecture principles to a real-world problem. The project achieved full implementation of a layered RESTful API with Node.js and Express, containerized with Docker, and orchestrated using Kubernetes. The Jenkins CI/CD pipeline ensured continuous integration and delivery, while Ansible playbooks automated infrastructure provisioning. Jest testing achieved the required 80% code coverage, and Prometheus/Grafana monitoring provided real-time visibility into system health. Scrum methodology guided the development process across two sprints, enabling structured and iterative delivery.

Several challenges were encountered during the project. Configuring Kubernetes probes with the correct port mapping required careful attention, as initial mismatches between the containerPort and the application's listening port caused deployment failures. Setting up Jenkins to authenticate with GitHub and push deployments to Kubernetes also required careful credential management. These obstacles were overcome through systematic debugging, documentation review, and iterative testing.

For further development, it is recommended to implement user authentication using JWT tokens to enable personalized recipe collections and contributions. The platform could also benefit from AI-powered recipe recommendation based on user preferences and cultural background. Additionally, integrating a mobile application (React Native) would significantly expand the platform's reach, particularly in regions with limited desktop access. Future work should also explore multi-language support to make the platform accessible to non-English-speaking communities across Africa.

REFERENCES

[1] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

[2] Newman, S. (2015). Building Microservices. O'Reilly Media.

[3] Schwaber, K. & Sutherland, J. (2020). The Scrum Guide. Scrum.org.

[4] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. ACM Queue.

[5] Express.js Documentation. (2024). https://expressjs.com/

[6] Jest Documentation. (2024). https://jestjs.io/

[7] Kubernetes Documentation. (2024). https://kubernetes.io/docs/

[8] Prometheus Documentation. (2024). https://prometheus.io/docs/

[9] Docker Documentation. (2024). https://docs.docker.com/

[10] Ansible Documentation. (2024). https://docs.ansible.com/